

Warm-up: the λ -calculus on paper, then in OCaml

Programming Paradigms
Session 4 — the warm-up, before the evaluator

Part 1 on paper, in pairs; Part 2 at the keyboard, with `lambda.ml`.

SOLUTIONS

The lecture's block 2 defined the calculus in three forms and one rule. Before we write the evaluator, we run the rule by hand. Nothing here is themed, and in Part 1 there is nothing to run. These are all we need:

x	$\lambda x. t$	$t u$	a variable; a function of x with body t ; an application
$f a b = (f a) b$,	$\lambda x. x y = \lambda x. (x y)$		application groups left; a body extends right
$(\lambda x. t) u \rightarrow t[x := u]$			the rule (β): the body, with u put in for every free x
normal order			the outermost redex first (leftmost among those)
applicative order			the innermost redex first (leftmost among those)
$\Omega = (\lambda u. u u) (\lambda u. u u)$,	$I = \lambda z. z$		two terms we meet again

Write one β -step per line and underline the redex you reduce.

Part 1 — on paper

Exercise 1 (β -reduction)

Reduce to normal form: $((\lambda x. \lambda y. x y) (\lambda z. z z)) (\lambda w. w)$.

Answer. $(\lambda x. \lambda y. x y) (\lambda z. z z) (\lambda w. w) \rightarrow (\lambda y. (\lambda z. z z) y) (\lambda w. w) \rightarrow (\lambda z. z z) (\lambda w. w) \rightarrow (\lambda w. w) (\lambda w. w) \rightarrow \lambda w. w$. Four steps; the last term has no redex. (In applicative order the second step would instead reduce inside the λy : $(\lambda y. y y) (\lambda w. w)$ — same normal form, another path.)

Exercise 2 (free, bound, capture)

In $(\lambda x. \lambda y. x y) y$, which occurrences of x and y are bound, and which are free? Reduce it. Be careful: the argument is a free y , and the body contains a λy .

Answer. x is bound by λx ; the y inside the body is bound by λy ; the argument y is free. Substituting y for x naively gives $\lambda y. y y$, in which the free y has been captured — wrong. Rename the binder first (α -conversion): $\lambda y'. x y'$, then substitute: $\lambda y'. y y'$, normal form, with the left y still free. This is the lab's `subst`, case λy with y free in the argument.

Exercise 3 (normal versus applicative order)

Reduce $(\lambda x. I) \Omega$ in normal order. Then start it in applicative order. What happens in each case, and why? (The lecture: normal order finds the normal form whenever one exists; applicative order may not.)

Answer. Normal order: the outermost redex is the whole term, and x does not occur in I , so $(\lambda x. I) \Omega \rightarrow I = \lambda z. z$ in one step. Applicative order must first reduce the argument: $\Omega \rightarrow \Omega \rightarrow \Omega \dots$ — the same term for ever, and the outer redex is never reached. The order did not change the answer; it decided whether we get one.

Exercise 4 (if as a function)

With `true` = $\lambda t. \lambda f. t$, `false` = $\lambda t. \lambda f. f$ and `if` = $\lambda c. \lambda t. \lambda e. c t e$, reduce `if true a b` to normal form, then `if false a b`. Under which order does this work as an `if` should — and what goes wrong under the other, when b is a term that loops?

Answer. $\text{if } \text{true } a \ b \rightarrow (\lambda t. \lambda e. \text{true } t \ e) \ a \ b \rightarrow (\lambda e. \text{true } a \ e) \ b \rightarrow \text{true } a \ b \rightarrow (\lambda f. a) \ b \rightarrow a$; likewise $\text{if } \text{false } a \ b \rightarrow *b$. In normal order the branch not chosen is never reduced, so a looping b does no harm. In applicative order both a and b are reduced before if chooses, and a looping b loops — block 1’s `my_if`. The repair by hand: pass the branches as functions of a dummy argument, $\lambda d. a$ and $\lambda d. b$, and apply the chosen one afterwards.

Exercise 5 (a number is a repetition)

With $1 = \lambda f. \lambda x. f \ x$ and $\text{succ} = \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$, reduce $\text{succ } 1$ to normal form. Which numeral is it, and why does the encoding make that term “two”?

Answer. $\text{succ } 1 \rightarrow \lambda f. \lambda x. f \ (1 \ f \ x) \rightarrow \lambda f. \lambda x. f \ ((\lambda x. f \ x) \ x) \rightarrow \lambda f. \lambda x. f \ (f \ x)$, which is 2: the numeral n is the function that applies f to x exactly n times, and succ adds one more application on the outside. A number is its own fold.

Part 2 — at the keyboard: `lambda.ml`

The type is the three forms; every function is one `match` on it. `dune exec ./main.exe` reports each hole as *todo*, then *pass* or *FAIL*.

	the function	what the checks want
W1	<code>size : term -> int</code>	the number of constructors; <code>size</code> $\Omega = 9$
W2	<code>free_vars : term -> string list</code>	sorted, no duplicates; $x(\lambda x. x)$ has x free
W3	<code>is_closed : term -> bool</code>	Ω is closed, $\lambda x. x \ y$ is not
W4	<code>subst : term -> string -> term -> term</code>	Exercise 2 in code: $(\lambda y. x)[x := y]$ is $\lambda y'. y$

W4 is the hard one: the `Lam (y, b)` case has two sub-cases, and `fresh` (provided) makes the new name. When W1–W4 pass, Part A of the lab is Exercise 3 in code — `step_cbn` and `step_cbv` on $(\lambda x. \lambda y. y) \ \Omega$.